C++ Templates

Generic Programming in C++

What Are Templates?

- A feature in C++ that allows functions and classes to work with any data type
- No need to write separate code for int, float, double, etc.
- One generic version handles all types
- This programming approach is called Generic Programming
- Parameters are of generic type and replaced by actual types at compile time

Function Templates

- Defined using the **template <typename T>** keyword
- T is a placeholder for any data type
- Syntax:

template <typename T>

return_type function_name(T parameter) { }

- The compiler automatically generates the correct version based on the type you pass

EXAMPLE CODE

```cpp
template <typename T>
T myMax(T x, T y)
{
    return (x > y)? x: y;
}

int main()
{
  cout << myMax<int>(3, 7) << endl;
  cout << myMax<char>('g', 'e') << endl;
  return 0;
}
```

Compiler internally generates and adds below code

```cpp
int myMax(int x, int y)
{
    return (x > y)? x: y;
}
```

Compiler internally generates and adds below code.

```cpp
char myMax(char x, char y)
{
    return (x > y)? x: y;
}
```

Class Templates

- Just like function templates, but applied to entire classes
- All member variables and methods can use the generic type T
- Syntax:

template <typename T>

class ClassName { };

- Allows a single class definition to store/operate on any data type

EXAMPLE CODE

```
#include <iostream>
using namespace std;

template <typename T>
class Storage {
   public:
      T data;

      Storage(T val) {
         data = val;
      }
      void display() {
       cout << "Stored value: " << data << "\n";
       }
};
int main() {
 Storage<int> s1(100);
 Storage<string> s2("Hello");

 s1.display(); // Output: Stored value: 100
 s2.display(); // Output: Stored value: Hello
}
```

Class Template: Multiple Types Example

- A class can use more than one template parameter: <typename T1, typename T2>
- Pair<string, int> → stores a name and age
- Pair<int, double> → stores a score and GPA
- Each pair can hold two different types simultaneously

CODE EXAMPLE

```
#include <iostream>
using namespace std;

template <typename T1, typename T2>
class Pair {
   public:
      T1 first;
      T2 second;
Pair(T1 a, T2 b) {
 first = a;
 second = b;
 }
 void display() {
 cout << "First: " << first << ", Second: " << second
<< "\n";
 }
};
int main() {
 Pair<string, int> person("Alice", 20); // name + age
 Pair<int, double> score(1, 9.5); // rank + GPA
 person.display(); // Output: First: Alice, Second:
20
 score.display(); // Output: First: 1, Second: 9.5

}
```

CODE EXAMPLE

```
#include <iostream>
using namespace std;

template <typename T1, typename T2>
void display(T1 a, T2 b) {
    cout << "First: " << a << ", Second: " <<
b << "\n";
}

int main() {
 display<string, int>("Alice", 20); //
Output: First: Alice, Second: 20
 display<int, double>(1, 9.5); // Output:
First: 1, Second: 9.5
 display<string, string>("Hi", "Bye"); //
Output: First: Hi, Second: Bye
}
```

Why Use Templates?

Code Reusability

Write once, works with many types

Type Safety

Errors caught at compile time, not runtime

Performance

No runtime overhead; compiler optimizes per type

Foundation of STL

vector, map, etc. are all built using templates

Thank you